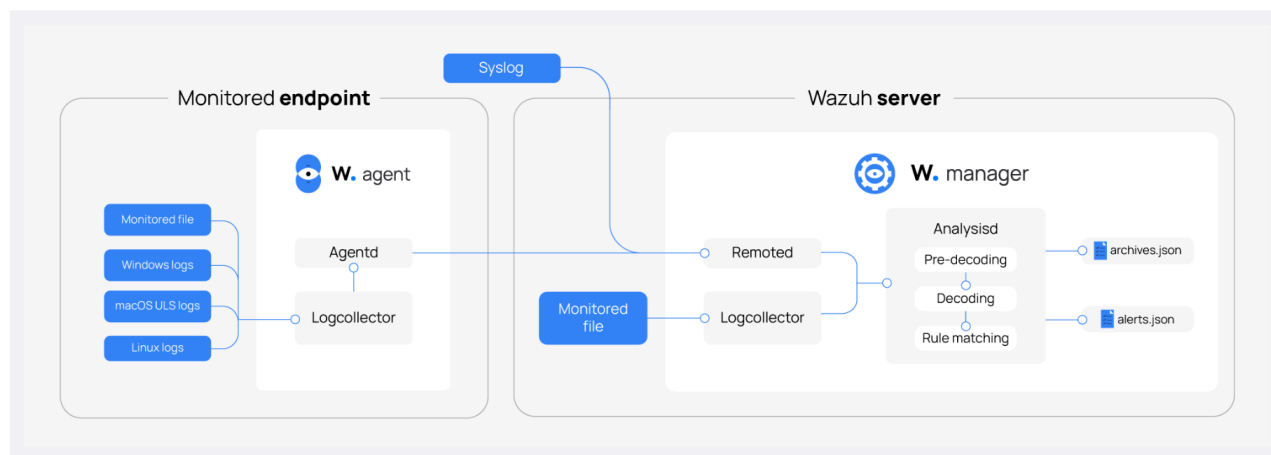


Log data collection



Phía Monitored Endpoint (bên trái)

Có hai luồng đầu vào song song vào Wazuh agent:

Luồng 1 — Qua Logcollector module: module **Logcollector** chịu trách nhiệm đọc trực tiếp từ 4 loại nguồn log:

- **Monitored file** — file log phẳng bất kỳ
- **Windows logs** — eventlog / eventchannel
- **macOS ULS logs** — Unified Logging System
- **Linux logs** — syslog, auditd, v.v.

Sau khi thu thập, **Logcollector** chuyển dữ liệu lên module **Agentd**, rồi **Agentd** gửi qua mạng đến Wazuh server.

Luồng 2 — Syslog trực tiếp (không cần agent): Các thiết bị như firewall, switch, router không cài được agent thì đẩy log thẳng lên Wazuh server qua **Syslog** (TCP/UDP port 514).

Phía Wazuh Server (bên phải)

Server có **hai điểm tiếp nhận log**:

Tiếp nhận từ agent → qua module **Remoted**, nhận gói tin từ Agentd của endpoint.

Tiếp nhận file được giám sát trực tiếp → qua **Logcollector** ở phía server, đọc "Monitored file" cục bộ trên server.

Sau đó cả hai luồng đi vào **Analysisd**, xử lý qua 3 bước tuần tự:

1. **Pre-decoding** — trích xuất metadata cơ bản (timestamp, hostname, program name) từ header log
2. **Decoding** — tìm decoder phù hợp, phân tách các trường dữ liệu có nghĩa
3. **Rule matching** — so khớp log đã decode với bộ rule để quyết định có sinh cảnh báo không

Hai đầu ra cuối cùng

File	Nội dung
archives.json	Toàn bộ log đã thu thập, kể cả log không kích hoạt alert nào
alerts.json	Chỉ các alert — log đã khớp với rule và có mức độ nghiêm trọng

Điểm quan trọng: `archives.json` lưu tất cả, đảm bảo không mất dữ liệu phục vụ điều tra sau này, dù log đó không tạo cảnh báo.

Cấu hình thu log Windows

Hai phương pháp thu log Windows

1. `eventchannel` — Khuyến nghị (Vista trở lên)

Hỗ trợ toàn bộ các kênh bao gồm Applications & Services logs, có thể lọc bằng XPath query.

Danh sách kênh được hỗ trợ sẵn:

Nguồn	Channel name	Mô tả
Application	Application	Quản lý ứng dụng hệ thống
Security	Security	Đăng nhập, tạo user/group, audit policy
System	System	Kernel và service control
Sysmon	Microsoft-Windows-Sysmon/Operational	Process, network, file changes
Windows Defender	Microsoft-Windows-Windows Defender/Operational	Scan, malware detection
McAfee	Application (provider: McLogEvent)	Kết quả scan McAfee
PowerShell	Microsoft-Windows-PowerShell/Operational	Audit PowerShell activity
Terminal Services	Microsoft-Windows-TerminalServices-RemoteConnectionManager	Remote desktop

Cấu hình cơ bản:

xml

```
<localfile>
  <location>Microsoft-Windows-PrintService/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Lọc theo điều kiện — XPath query:

Chỉ lấy event có Level ≤ 3 (Warning trở lên):

xml

```
<localfile>
  <location>System</location>
  <log_format>eventchannel</log_format>
  <query>
    \<QueryList\>
      \<Query Id="0" Path="System"\>
        \<Select Path="System"\>*[System[(Level<=3)]]\</Select\>
      \</Query\>
    \</QueryList\>
  </query>
</localfile>
```

Chỉ lấy Event ID cụ thể (ví dụ 7040):

xml

```
<localfile>
  <location>System</location>
  <log_format>eventchannel</log_format>
  <query>Event/System[EventID=7040]</query>
</localfile>
```

2. eventlog — Tương thích mọi Windows version

Chỉ giám sát được **Application**, **System**, **Security** — không hỗ trợ Applications & Services logs.

xml

```
<localfile>
  <location>Application</location>
```

```
<log_format>eventlog</log_format>
</localfile>
```

Ruleset Windows Event Channel

Wazuh có bộ rule riêng cho từng kênh, phân chia theo Rule ID range:

Nguồn	Rule ID	File rule
Base rules	60000–60099	0575-win-base_rules.xml
Security	60100–60599	0580-win-security_rules.xml
Application	60600–61099	0585-win-application_rules.xml
System	61100–61599	0590-win-system_rules.xml
Sysmon	61600–62099	0595-win-sysmon_rules.xml
Windows Defender	62100–62599	0600-win-wdefender_rules.xml
McAfee	62600–63099	0605-win-mcafee_rules.xml
PowerShell	91801–92000	0915-win-powershell_rules.xml
Others (generic)	64100–64599	0620-win-generic_rules.xml

File base 0575-win-base_rules.xml chứa toàn bộ parent rule cho các kênh được giám sát mặc định.

Phân tích Log trong Wazuh — 3 Giai đoạn

Lấy log mẫu SSH làm ví dụ xuyên suốt:

```
Feb 14 12:19:04 192.168.1.1 sshd[25474]: Accepted password for Stephen from 192.168.1.133 port 49765 ssh2
```

Giai đoạn 1 — Pre-decoding

Engine đọc **phần header** của log theo chuẩn syslog, trích xuất 3 trường cố định:

```
timestamp:    'Feb 14 12:19:04'  
hostname:     '192.168.1.1'  
program_name: 'sshd'
```

Giai đoạn này không cần cấu hình — Wazuh tự động nhận dạng cấu trúc syslog chuẩn.

Giai đoạn 2 — Decoding

Engine tìm **decoder phù hợp** trong bộ decoder có sẵn. Với log SSH trên, hai decoder trong file `0310-ssh_decoders.xml` được áp dụng theo thứ tự cha → con:

Decoder cha — khớp theo `program_name`:

xml

```
<decoder name="sshd">  
  <program_name>^sshd</program_name>  
</decoder>
```

Decoder con — trích xuất thông tin cụ thể từ nội dung log:

xml

```
<decoder name="sshd-success">  
  <parent>sshd</parent>  
  <prematch>^Accepted</prematch>  
  <regex offset="after_prematch">^ \S+ for (\S+) from (\S+) port (\S+)</regex>
```

```
<order>user, srcip, srcport</order>
</decoder>
```

Kết quả trích xuất được:

```
dstuser: 'Stephen'
srcip:   '192.168.1.133'
srcport: '49765'
```

Cơ chế hoạt động của decoder con:

- `prematch` — kiểm tra log có bắt đầu bằng `Accepted` không
- `regex` — dùng capturing group (`\S+`) để bắt các giá trị
- `order` — ánh xạ thứ tự group vào tên trường tương ứng

Giai đoạn 3 — Rule matching

Engine so khớp log đã decode với toàn bộ **ruleset**. Rule `5715` trong file `0095-sshd_rules.xml` khớp:

xml

```
<rule id="5715" level="3">
  <if_sid>5700</if_sid>
  <match>^Accepted|authenticated.$</match>
  <description>sshd: authentication success.</description>
  <group>authentication_success,pci_dss_10.2.5,</group>
</rule>
```

Giải thích các trường trong rule:

Trường	Ý nghĩa
<code>id="5715"</code>	ID duy nhất của rule
<code>level="3"</code>	Mức độ nghiêm trọng (1–15)
<code>if_sid="5700"</code>	Chỉ kích hoạt nếu log đã khớp rule cha 5700 trước
<code>match</code>	Pattern khớp nội dung log
<code>group</code>	Nhóm phân loại, bao gồm tag compliance (PCI DSS)

Ngưỡng sinh alert: Wazuh mặc định chỉ tạo alert khi `level > 2`. Rule này có `level=3` → **alert được tạo** và hiển thị trên dashboard.

Mở rộng — Custom decoder và rule

Khi gặp log từ ứng dụng không được hỗ trợ sẵn, có thể tự viết:

- **Custom decoder** — định nghĩa regex để trích xuất trường mới
- **Custom rule** — định nghĩa điều kiện và mức độ alert cho log đó

Đây là điểm mạnh của Wazuh: hoàn toàn mở rộng được mà không cần thay đổi core.

Cơ chế thu log Windows của Wazuh — Sâu ở mức kỹ thuật

1. Windows API được gọi — `winevt.h`

Cụ thể các Win32 API được gọi:

API	Mục đích
<code>EvtSubscribe()</code>	Đăng ký nhận event theo real-time từ một channel
<code>EvtQuery()</code>	Truy vấn event theo XPath hoặc structured XML query
<code>EvtNext()</code>	Lấy batch event tiếp theo từ kết quả query

API	Mục đích
<code>EvtRender()</code>	Render event handle thành XML string
<code>EvtFormatMessage()</code>	Lấy trường Message từ provider (cần ProviderName)
<code>EvtClose()</code>	Đóng handle sau khi xong

Wazuh agent tích hợp với Eventchannel API — chính là Windows Event Viewer API phía dưới — để truy cập lập trình vào system, application và security logs. Hiện tại agent format event thành JSON, bao gồm cả event data dạng message lẫn full XML string

2. Cơ chế subscription — push model

Wazuh dùng callback `event_channel_callback(EVT_SUBSCRIBE_NOTIFY_ACTION action, os_channel *channel, EVT_HANDLE evt)` — đây là push model: Windows tự gọi callback của Wazuh mỗi khi có event mới, thay vì Wazuh phải polling liên tục.

[GitHub](#)

Wazuh Agent 5.0 thiết kế rõ ràng hơn với các class: `Session` quản lý ETW session, `Provider` đại diện ETW provider, `Consumer` xử lý event nhận được, `BookmarkManager` lưu/đọc bookmark, `QueryBuilder` xây dựng XPath query. Hệ thống dùng bookmark để track vị trí đọc cuối — khi agent restart sẽ tiếp tục từ đúng vị trí đó, không mất event.

3. Các field được thu — hai nhóm chính

Module thu được **hai nhóm field chính** là `System` và `EventData`. `System` tương đối tĩnh, chứa thông tin về môi trường; `EventData` hoàn toàn động, chứa các field bổ sung tùy từng loại event. [GitHub](#)

Nhóm `System` — luôn có:

Các field tĩnh trong `System` gồm: `ProviderName`, `ProviderGuid`, `EventID`, `Version`, `Level`, `Task`, `Keywords`, `SystemTime`, `EventRecordID`, `Channel`, `Computer`. Ngoài ra có các field không phải lúc nào cũng xuất hiện: `EventSourceName`, `Opcode`, `ProcessID`, `ThreadID`. [GitHub](#)

Hai field đặc biệt — không lấy từ XML:

Message và SeverityValue **không thể lấy từ XML string**. SeverityValue được tính từ keywords và level. Message được lấy qua hàm get_message() trong read_win_event_channel.c, dùng ProviderName làm input để gọi EvtFormatMessage().

[GitHub](#)

Nhóm eventdata — động theo từng event:

EventData có thể rỗng; các tham số của nó có thể có tên không mang thông tin rõ ràng, ví dụ "Data": "10.0.0.3" — nhưng tất cả đều có thể lấy từ XML. [GitHub](#)

4. Format JSON gửi lên server

Decoder windows_eventchannel được nhúng thẳng vào source code C của Wazuh, **không phải file XML decoder** như các decoder khác — đó là lý do không tìm thấy file decoder này trong thư mục cài đặt. [Google Groups](#)

Event sau khi được EvtRender() thành XML sẽ được wrap vào JSON theo cấu trúc:

json

```
{
  "win": {
    "system": {
      "providerName": "Microsoft-Windows-Security-Auditing",
      "eventID": "4688",
      "level": "0",
      "task": "13312",
      "keywords": "0x8020000000000000",
      "systemTime": "2024-01-01T00:00:00Z",
      "eventRecordID": "12345",
      "channel": "Security",
      "computer": "HOSTNAME",
      "severityValue": "AUDIT_SUCCESS",
      "message": "A new process has been created..."
    }
  }
}
```

```
    },  
    "eventdata": {  
      "subjectUserName": "john",  
      "subjectDomainName": "CORP",  
      "newProcessName": "C:\\Windows\\cmd.exe",  
      "commandLine": "cmd.exe /c whoami"  
    }  
  }  
}
```